

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

<Dissertation Title>

<Fernando Azeredo>

WORKING VERSION



Mestrado em Engenharia Informática e Computação

Supervisor: Prof. <António Sá Pinto>

June 18, 2026

Abstract

Music Information Retrieval (MIR) visualization tools play a crucial role in music analysis, annotation, and interactive music applications. Accurate visual representation of audio and algorithmic outputs is essential for the evaluation, analysis, and annotation of MIR tasks. This thesis develops LayerIt, a Python-based object-oriented (OO) tool that combines established MIR resources. The developed project emphasize modularity and reusability, the design defines clear object boundaries for data sources, score aligned visualization layers, simplifying the addition of new data types and comparison modes. Visuals are exported as SVG to allow further customization and future integration with web or interactive tools. As a validation case, we use LayerIt to visualize and compare beat tracking outputs, demonstrating how score aligned visualizations support analysis and evaluation of annotated and algorithmic MIR data. The project is presented as a foundation for further development toward a mature framework for specialized MIR visualizations.

Keywords: keyword1 • keyword2 • keyword3 • keyword4

Acknowledgements

<Fernando Azeredo>

Contents

List of Figures

List of Tables

List of Acronyms

Chapter 1

Introduction

1.1 Contextualization

Music Information Retrieval (MIR) is a research field that develops computational methods to analyze, interpret, and retrieve musical data from performance recordings. MIR is heavily reliant on data, and thus, visualization plays a crucial role; how and what we visualize in data can shape not only the conclusions one makes, as well as providing guidance on how to further progress. The problem of visualization in MIR is especially critical because music is inherently subjective and abstract. Music is also a uniquely auditory medium, translating it into visual form is a challenge that spans millennia, with many attempts at "musical written" languages throughout history. Yet this task remains unfolding. For MIR researchers, this means working with diverse data representations, each capturing different aspects of music.

1.2 Developed Project

For this thesis we developed LayerIt, a tool that combines multiple state-of-the-art and well established MIR resources into one single Python-based object-oriented (OO) tool. LayerIt takes advantage of ScoreWarp, librosa, matplotlib, and BeatThis, in order to produce score aligned audio visualizations with overlaid algorithm outputs. This tool is meant

1.3 Motivation

1.4 Thesis Structure

1.5 Thesis Structure

Chapter 2

Literature Review

2.1 Introduction

Music Information Retrieval (MIR) is the field of research that focuses on the development of algorithms and systems to analyze, retrieve, and organize music data. MIR bridges a wide range of disciplines, including: musicology, signal processing, machine learning, and human-computer interaction. [mullerFundamentalsMusicProcessing2015]

2.2 Visualization Types

We can sum up all music related visualizations into three categories:

1. Sheet Music and Score Images
2. Symbolic Representations
3. Audio Representations

2.2.1 Sheet Music and Score Images

Sheet music describes musical work using a formal language based on musical symbols and letters, thus, this kind of music representations always require a level of music literacy from the performing musician. Furthermore sheet music usually does not induce explicit information on how a given piece should be performed, it is intended for the musician to have a certain level of interpretation liberty. This can include the variance of tempo, articulation and dynamics among others.

Sheet music also comes in a variety of forms that vary throughout History and cultures. For our purposes we are going to focus on the western standard of music notation. Still, within western music, there can be vastly different forms of notation, however, most are based on a staff which sets five horizontal lines and four spaces, each representing a different musical pitch. Musical relevant symbols are put inside the staff to denote pitch, temporal and expressive indications. [mullerMusicRepresentations2015a]

2.2.2 Symbolic Representations

Symbolic Representations are machine-readable formats where musical events and expressions are explicitly encoded. These come in various formats, such as MIDI and MusicXML.

Visually we can identify two main applications for this category, these include: piano-roll representations and digital score representations.

Historically piano-roll representations come from so-called “player pianos”, these were self playing pianos that became quite popular in the late 19th to early 20th century. This was achieved by a mechanism that would hit keys of the piano through a continuous roll of paper with perforations. The roll would move over a reading system (known as a tracker bar) that would trigger the musical actions accordingly.

This concept of a continuous and explicit representation of music performance would later prove useful for digital applications. Such is the case for the Piano-Roll.

The **Piano-Roll** is a staple on any DAW as a visual and interactive tool to write and edit musical events in the form of MIDI notes and attributes (such as velocity and other expressive controls).

MusicXML refers to the XML-based language to transcribe score music into the digital domain. There are multiple sub-variants of the MusicXML format for specific applications, however all share an XML based language underneath.

MusicXML was developed to serve as a universal format for storing and sharing music scores across different music notation applications. Music notation digital editors then employ proprietary variants of this format for addressing the program’s specific characteristics. In some cases these may not even be xml based, but MusicXML format can be considered the universal format. These include, among others:

- **.mscz and .mscx**: These are MuseScore specific formats named for "MuseScore Format" and "Uncompressed MuseScore" format respectively. Both xml based.
- **.dora** - "Dorico" specific format (non xml based)
- **.sib** - "Sibelius" proprietary format of company Avid (also non xml based)
- **.mei** - The "Music Encoding Initiative" (MEI). XML based.

The .mei format is the one that is essential for this thesis and the most useful for MIR purposes. MEI stands for both the XML-based score format in itself (.mei), and the community that develops it. Citing from the MEI website’s introduction:

“The Music Encoding Initiative (MEI) is a community-driven, open-source effort to define a system for encoding musical documents in a machine-readable structure. MEI brings together specialists from various music research communities, (...) to define best practices for representing a broad range of musical documents and structures.”[**MEI-Website**]

Important to note that MEI focuses on a "machine-readable structure". This structure relies on unique identifiers (@xml:id) that make every element in the score individually addressable,

from individual noteheads to complex spanning elements like slurs and ties. However, more than a music score format, MEI also serves to keep various notation, performance and interpretation data not only stored but also linked to individual score elements. This is why the MEI format and its community are so essential to MIR as a whole.

That being said, MEI is a purely data driven project, and thus in order for it to be visualized or interacted with it needs to be decoded into a "human-readable format". That's where Verovio comes in. The MEI and Verovio projects are closely inter-linked for that reason. While MEI defines how musical information is stored, Verovio is the primary tool used to transform that data into interactive visualizations and performance-ready outputs.

Verovio is a comprehensive open-source library designed to serve as a native typesetting engine for the MEI format. Verovio's primary role is to engrave MEI data directly into SVG without losing rich metadata through intermediate format conversions. The choice for making it into an SVG is an essential one for multiple reasons, quoting from Verovio's reference book:

"In a web environment, the addressability can be used for highlighting graphical elements such as notes or any other music symbols. One additional characteristic of SVG is that its XML tree can be structured as desired, and an innovative design feature of Verovio was to go further in the structuring of the output by leveraging this characteristic. Since Verovio implements the MEI structure internally, this key feature of SVG made it possible to preserve the MEI structure in the design of the SVG output in Verovio. Preserving the MEI structure in the SVG output is a considerable overhead in the rendering process but makes it a unique feature of Verovio.

"As a result, Verovio output in SVG is not the end of a unidirectional rendering process. Quite on the contrary, it should instead be seen as an intermediate layer standing between the MEI encoding and its rendering that can act as the cornerstone for a bi-directional interaction: from the encoding to the notation, but also from the notation to the encoding through the user interface." repertoireinternationaldessourcesmusicalesVerovioReferenceBook2021

To further demonstrate the usefulness of the MEI and Verovio, I'll present next a quote from the article relative to Piano Precision (PP) a GUI software for audio-to-score visualization, we will explore this tool further in the next section.

"Technically, what makes MEI especially useful is that it can be rendered to graphical SVG output while retaining its semantically meaningful XML hierarchy and unique identifiers for every score element (e.g., ties, notes). Piano Precision renders the scores into SVG output using Verovio, an open-source toolkit for visualizing MEI data. This makes it easy to locate interesting objects in a score and, e.g., highlight them in the rendering. The result is interactive images in which users can flip pages, zoom in and out of a score, and click on elements in a score to jump to their playback positions. " jiangPIANOPRECISIONADVANCING2025

2.2.3 Audio Representations

Note: The figures in this chapter will follow the same music excerpt from Chopin Op10 n3. They were generated using the AudioVisualizers.py script, which is available in the appendix.

Audio representations are graphs and/or plots that intend to represent sound. These may not be for strictly representing music, the intent is to translate sound from the audible to the visual domain. Audio is a term that refers to an acoustic sound that is turned into an electrical signal. Most audio representations, are thus digitally generated through applying mathematical equations to the audio signal, such as the Fourier Transform.

We can infer at least 3 components that define a given audio. These are, time, frequency and amplitude/energy. An audio visualization will present explicit information of at least 2 of these.

Waveforms are the most common and one of the simplest ways to represent audio. It consists of a single line on a graph in a 2D graph where amplitude is the y-axis and time is in the x-axis. This representation does not provide frequency information, it only shows how amplitude of the audio changes through it's time.

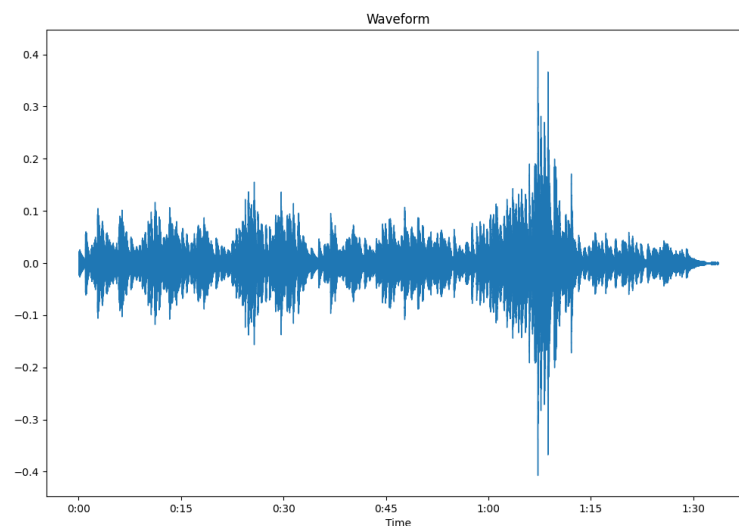


Figure 2.1: Example of an audio waveform.

A spectral visualization, or a spectrum, refers to a graph with only frequency and amplitude/energy representation in a given time window.

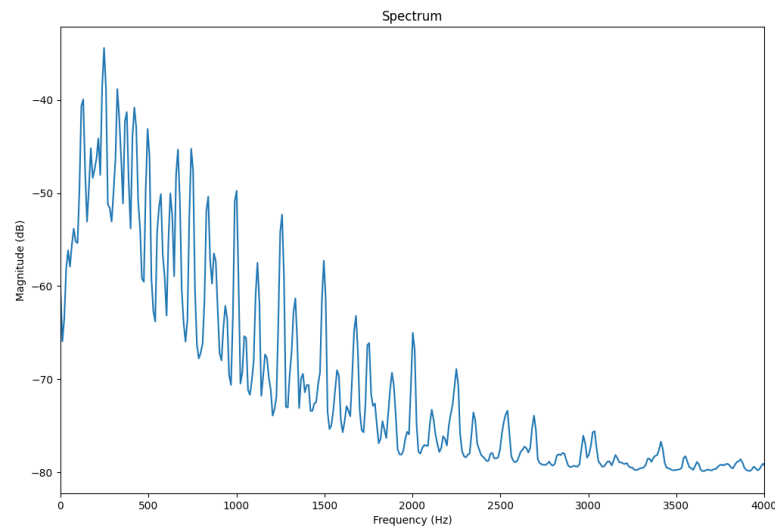


Figure 2.2: Example of audio Spectrum.

For displaying the three sound components the plot needs to represent three distinct dimensions. The spectrogram achieves this by having on the y axis the frequency, time on the x-axis and color as substitute for the third axis (amplitude), and in this way displaying all three parameters in a single 2D visual plot. There are multiple weights and variances of the spectrogram that have different applications.

The way frequencies are displayed in a given spectrogram can help understanding audio better for certain situations. There can be a wide variety of spectrogram representations, that can vary in how it's contents are displayed.

The linear spectrogram is the most basic form of spectrogram, where frequencies are displayed in a linear scale. Meaning, frequencies are equally spaced on the y-axis. For music applications, this kind of representation tends to be less useful, as it does not reflect the way humans perceive sound.

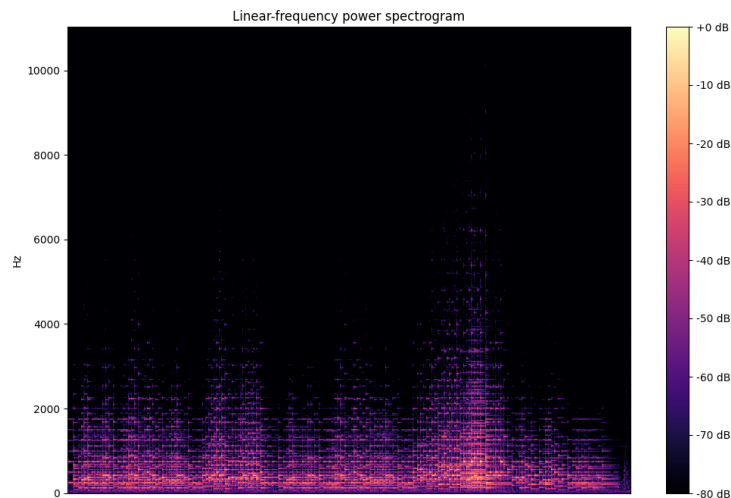


Figure 2.3: Example of Linear spectrogram.

The logarithmic spectrogram displays frequencies in a logarithmic scale. This mimics better the way humans perceive sound.

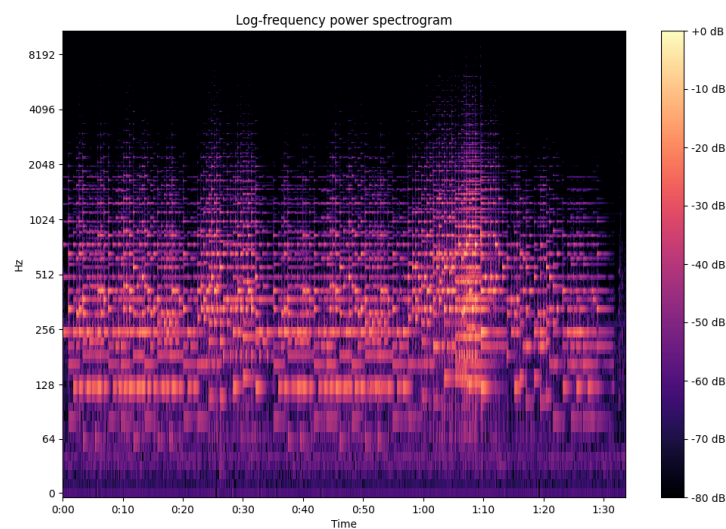


Figure 2.4: Example of Logarithmic spectrogram.

The Mel spectrogram also displays frequencies taking into account the human hearing curve. The difference being, that log spectrogram simply organizes frequencies logarithmically. Whilst the Mel spectrogram organizes frequencies according to the Mel scale, which is a perceptual scale of pitches judged by listeners to be equal in distance from one another. For music related applications this type of spectrogram is much more useful, since we can more clearly distinguish in

between musical pitches.

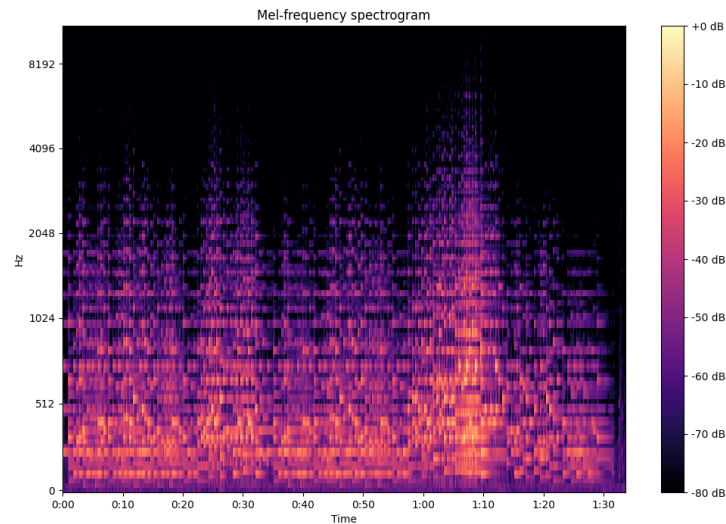


Figure 2.5: Example of Mel spectrogram.

Next I'll introduce the Constant-Q Transform (CQT) and the Chromagram. These are, to some extent, visual representations that derive from the spectrogram, the difference being that whilst the later shows frequency data, the CQT and chromagram can be used to visualize note data. To be exact, tone height in the case of the CQT and Chroma for the Chromagram.

“(...) a pitch can be separated into two components, which are referred to as tone height and chroma. The tone height refers to the octave number and the chroma to the respective pitch spelling attribute contained in the set {C, C $\sharp$, D,(...), B}.” [mullerFundamentalsMusicProcessing2015]

The CQT organizes its frequency content using geometrically spaced frequency bins. This is made in order to resemble the equal-tempered system. More commonly, the CQT can be interpreted as a Piano-Roll like representation.[valleVisualDisplayRetrieval]

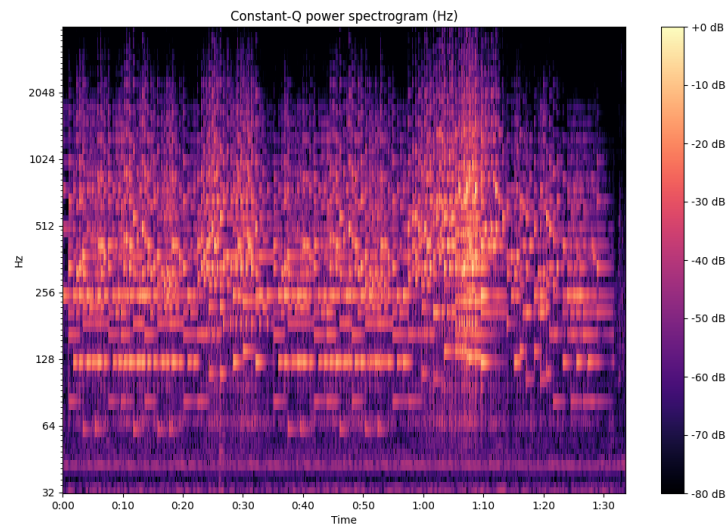


Figure 2.6: Example of a CQT.

While the CQT can distill the frequency spectrum into bins that correlate to individual notes, the chromagram takes this a step further by visually summing the energy of each note across all octaves. Thus in the end we get a plot divided into 11 lines, each representing a give note. In it's essence this type of visualization serves to see the usage of the same notes across the recording. A chromagram can be useful for determining harmonic and scale related data.

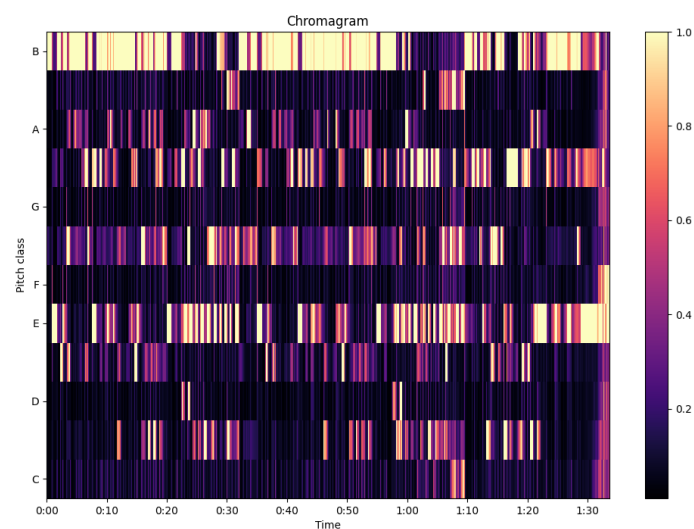


Figure 2.7: Example a Chromagram.

[mullerFundamentalsMusicProcessing2015, jungUnifiedCrossmodalTranslation2026a, khodzhaevPract
valleVisualDisplayRetrieval]

2.3 MIR Visualization Tools

Within MIR we can identify two main ways for visualization of audio for MIR tasks, these are either by using GUI based tools, or by using programming-based workflows.

For GUI based tools two stand-out, these being Audacity and especially Sonic Visualizer (SV). Audacity is one of the most popular audio editors; it is designed for general audio related tasks from audio editing, recording, processing, and visualization. On the other hand, SV is a strictly designed program for audio visualization and analysis. Whilst Audacity the most "user friendly" and popular program, SV is meant for scientific and specialized audio visualization tasks.

Important to note that both software solutions are free and open-source based. These are important features, since they allow for users to easily access them, but also for researchers and experts to modify them and thus, contribute to the development of SV and new SV-like tools.

One such example is Piano Precision, a SV based GUI tool for visualizing score-aligned spectrograms.

"The Piano Precision UI is adapted from Sonic Visualizer, with an additional area displaying a score (...) The interface is designed to help streamline the processes of loading a digital score and a performance recording, editing and visualizing annotations (...), and exporting score-aware annotations. "[jiangPIANOPRECISIONADVANCING2025]

While modern GUI-based visualization tools may be powerful, they might not provide features that are required for state-of-the-art research and development. MIR research often requires highly specialized visualizations tailored to specific research questions that pre-built tools simply do not support. Understandably, the development of these programs simply cannot keep up with the constantly evolving advancements in the field.

That is why it is also quite common for researchers to develop programming-based methods for creating specialized visualizations. Researchers might need to integrate multiple data types (audio, scores, annotations, computational outputs) in ways that GUI tools cannot replicate.

Moreover, code-based workflows can be versioned, shared, and included in publications, ensuring reproducibility this is an essential requirement for academic research.

Another argument is that MIR is increasingly geared towards machine learning solutions and algorithms. The need to work with large music datasets and integrate machine learning applications for training, feature extraction, and other related tasks, explains the reliance on code-based visualization workflows.

Furthermore, the fact that MIR is fundamentally a computational research field more broadly justifies this approach.

Currently, Python is the go-to language for MIR visualization tasks. Key libraries such as Librosa, Matplotlib, and Madmom, along with APIs such as Modusa and tools like Jupyter Notebooks, are common in MIR research.

2.4 Audio-to-Score Alignment

The task of Audio-to-Score Alignment (score matching or score alignment) consists of “synchronizing an audio recording of a musical piece with the corresponding symbolic score” carabias-ortiAUDIOSCOREALIGNMENTa. There are multiple applications for such a feature, from automatic score following for music performance or for guided listening, to interactive navigation between audio and score (useful for ex. audio editors) to MIR research and development.

There are two modes of alignment: offline alignment and online alignment. Offline alignment, consists of accessing a musical recording and performing the alignment afterwards, this can be useful for tasks such as musical analysis and audio editing. Online alignment (also known as also known as score following), processes the data in real-time while the audio signal is being received, this is useful for tasks like autonomous accompaniment of the score while a performance is occurring. Offline alignment is able to use the full length of the recording in order to determining the optimal alignment, this is because it can “look into the future” in order to establish the optimal matching. Because of this, Online alignment can’t provide the same level of detail unless some deliberate latency is introduced into the system thus, online modes need to take into account precision trade-offs.

Besides online and offline modes of score matching, the alignment in itself can be done to various levels of the performance or score these include:

- Note-level
- Beat-level
- Segment-level

Note-level alignments are score matching processes that aim to synchronize at the individual note level. At it’s core, note-level alignments are systems based on note and onset/offset detection and linking. In MIR literature it can be also referred as “fine-grained alignment” due to the level of precision that these methods require. Contemporary Audio-to-Score MIR research focuses on this type of alignment not only because of the precision element but also because it can retrieve data that is relevant for other MIR fields and tasks. However, pure note-level alignments might falter if the performance deviates from the score i.e. if the performer skips, repeats or adds notes and sections of the score. That’s why some score matching solutions try to employ, on top of fine-grained alignments, other methods.

Piano Precision (PP) software provides a note-level score aligned visualization. While a recording is playing PP shows the current playback position via highlighting the element being played in the score note by note. This enables annotations to be mapped to both the audio and individual score elements simultaneously. The authors note the usefulness of this tool for performance researchers. [jiangPIANOPRECISIONADVANCING2025]

Beat-Level alignments aim to follow along beat and/or measures dictated in the score, and then linking them to the performance. Early score aligners (ex.duanSoundprismOnlineSystem2011)

would rely on such methods. Today, score alignment solutions, although mostly are note-level types, still combine Beat-Level alignment layers within their algorithms. A good example of this comes from [peterAutomaticNoteLevelScoretoPerformance2023], in it the authors improved on other alignment models, one strategy employed was adding analysis of “alignments at both the beat and the bar level”. The beat level alignment would serve as “anchor points” in order to prevent “cascading errors, as misalignment will not propagate beyond the next anchor point”, they noted that this improved results of automatic alignments.

Segment type alignments, map broad portions of performance audio to corresponding visual blocks in the score. This type of score matching is mostly made manually, it is common to see it in follow-along videos to help viewers listen to the music while visualizing the score. This type of alignment is considered “weak but musically meaningful” jungUnifiedCrossmodalTranslation2026a since segments can be deliberately chosen i.e. to better show melodic or harmonic structure.

ScoreWarp is a relevant visual score alignment tool for this thesis. It presents a score that is tied to a physical time axis where, each element of the score is “physically shifted so that their location linearly coincides with the physical time of their sounding” goeblLetsScoreWarpAgain2025a. ScoreWarp takes advantage of the MEI format and Verovio in order to automatically shift the position of musical elements, while maintaining the semantic integrity of the original score. In the next chapter we are going to explore this tool more in depth, since it became one of the key tools for the development of the project for this thesis.

2.5 Summary

This chapter provided a comprehensive overview of Music Information Retrieval (MIR) visualization techniques and tools, establishing the foundation for understanding audio-to-score alignment.

We began by categorizing music visualizations into three main types: traditional sheet music representations, symbolic machine-readable formats, and audio-based representations. With this we established the importance of the MEI and Verovio as well as the usage of the SVG visual format in this context.

We then examined the contemporary landscape of MIR visualization tools, distinguishing between GUI-based solutions such as Sonic Visualizer and Piano Precision, which provide user-friendly interfaces for audio analysis, and code-based workflows that afford researchers greater flexibility to develop specialized visualizations tailored to specific research questions. Python-based approaches using libraries like Librosa and Matplotlib have become the de facto standard for academic MIR work, particularly due to their integration with machine learning pipelines and reproducibility requirements.

Finally, we introduced the core task of this thesis: Audio-to-Score Alignment. This process synchronizes musical performances with their corresponding symbolic scores. ScoreWarp emerged as a particularly relevant tool, demonstrating how MEI and Verovio can be leveraged to

create visually warped scores that maintain semantic integrity while aligning with performance time.

These foundations establish the conceptual and technical backdrop for the practical work presented in subsequent chapters.

Chapter 3

Developed Solution

3.1 Introduction

3.2 Developed Iterations

This thesis started with the proposed title “Interactive Musical Beat Annotation GUI”. The initial thesis proposition was to evaluate modern GUI software solutions used for beat annotation (BA) (such as SV), and build upon them with a focus in beat annotation practices. The proposed goals included:

1. Either develop a web-based annotation system, or advancing the existing SV framework. Emphasizing collaborative features, real-time feedback and build on interaction mechanisms for beat annotation.
2. Integration of ML self-supervised and human-in-the-loop features that provide intelligent annotation suggestions and improve user experience. System inspired on [yamamotoHumanintheLoopAdaptationInteractive2021].
3. Validation through user studies.
4. Publish the developed work as an Open Source Python library/framework. The purpose being, to contribute to MIR research community for further development and integration with other existing frameworks.

The motivation from this proposal came from mostly two identified flaws in this topic: The first is that, although the performance of beat tracking algorithms has been improving significantly in recent years, they mostly rely on deep learning based approaches. The issue with this, is that in order for these systems to be trained they need increasingly large and more complex datasets that, although not always directly (peterAutomaticNoteLevelScoretoPerformance2023), need to some extent human made and/or verified beat annotations. The second one is related to human made beat annotation methods, that for the most part, still is a extensive and time consuming task.

Besides that, although SV is a powerful tool, it presents a clear gap for beat annotation, where the integration of newer SOTA BT algorithms is difficult and there have been little to no new added features that improve BA task significantly.

Summarizing, the thesis came from a focus in BT, BA and interactive GUIs to, in the end, focusing on a purely MIR visualization problem. Next we will present different iterations the development phase undertook. This should contextualize the change in scope and motivation of the thesis.

3.2.1 BeatMapJS

Developed from October 2025 until February 2026, BeatMapJS was our first attempt to create a GUI application for beat annotation. Our goal was to experiment and explore interactive BA features, libraries and frameworks while focusing on a web-based solution. Using JS/HTML, BeatThis, and Wavesurfer.JS, we successfully built an interactive spectrogram with a web-based interface that integrated the BeatThis algorithm. The application included beat annotation basic tools, region annotation capabilities, a table for editing tempo annotations and descriptions, and the ability to import and export beat annotations to .txt files (compatible with SV).

Although the program was functional, it was ultimately abandoned. The main reason was that it provided no significant innovative BA features beyond BeatThis integration, and critically, no Python-based library was produced as the backbone of the project, which was an important goal for ensuring the work could be reused by the community.

3.2.2 MEI-Basic-Edit

Running from February until March 2026, this project aimed to create a GUI ecosystem where users could manually adjust score elements while simultaneously viewing an audio visualization layer. We had identified an innovative BA feature, Audio-To-Score Alignment, and hypothesized it would greatly benefit both beat annotation and evaluation tasks. The project served as a learning opportunity to explore MEI and Verovio frameworks along with ScoreWarp.

Implemented in JS/HTML using Verovio, MEI score format, and SVG, the application allowed users to upload a recording and its corresponding MEI score file. A static spectrogram would be generated with the score displayed horizontally below it. Users could interact with individual score elements by clicking and dragging them horizontally to manually align the score with the audio-visual representation. The system worked by rendering both the score and spectrogram into an SVG, allowing direct interaction with the SVG file. The GUI also included an integrated XML text editor for direct manipulation of the SVG score code.

This experiment revealed significant complexity in creating interactive scores—both from UI/UX and technical perspectives. However, it provided invaluable insights into how Verovio and MEI function, as well as SVG structure, knowledge that would prove essential for the developed end solution. The project was ultimately abandoned as it did not produce a reusable Python library and the interaction design was functional enough to justify further development in this direction.

3.2.3 BeatPrecision, a PP fork

In March 2025, we explored building on the PP and SV frameworks to create a BA GUI tool. The rationale was straightforward: PP already provided an embedded score alignment feature and all SV functionality, so adding a few BA-focused improvements seemed like a viable path to developing a relevant annotation tool. Using Qt/C++, we integrated BeatThis and implemented automatic score follow-along in a horizontal manner to synchronize with SV’s spectrogram and audio-visualizations.

While functional, the project revealed significant challenges. The SV/PP codebase proved complex and difficult to manage. Long compile times and the sheer size of the codebase made each iteration increasingly challenging to implement, manage and debug. More importantly, this iteration taught us a critical lesson: we needed to distinguish between the interaction and visualization problems in BA. We realized we could not tackle both simultaneously and would need to choose a direction. Recognizing a clear gap in the field, we opted to focus on the visualization aspect, as we saw greater potential for the thesis to benefit the community in this area.

3.2.4 Score-Alignment.py

By the end of March 2025, we shifted our focus to creating a Python-based visualization tool with score-aligned spectrograms. Our motivation was to concentrate on score alignment in a Python based environment, while exploring the Modusa framework’s capabilities. Using Python, Modusa, and matplotlib, we built an interactive spectrogram.

However, this project was abandoned relatively quickly. Implementing a truly interactive and fluid spectrogram (with zoom, pan, timeline, and playback cursor) proved significantly more challenging than anticipated. While we managed to develop a functional interactive spectrogram, its implementation was clunky and unsatisfactory, leading us to discontinue the project in favor of other approaches.

3.2.5 Score-Alignment.js

In April 2026, we attempted to recreate the last iteration, this time implementing it in JS/HTML. Building on the experience from MEI-Basic-Edit and the initial BeatMapJS project, we aimed to create a comprehensive audio-to-score alignment with spectrogram visualization. Leveraging JS/HTML, Verovio/MEI, and Wavesurfer.js, we developed a system that displayed MEI scores horizontally alongside spectrograms. Timestamps—intended to represent beat annotations—could be directly stored in the MEI file using the `<when>` function, and notes in the score would highlight when hovered over, selected, or associated with a timestamp.

While not fully functional, the application showed promise. However, we made a deliberate decision to shift direction once again. We recognized the need to develop a strictly visualization tool purely in Python with an emphasis on an Object-Oriented approach that could be reused and extended by the community. This pivotal decision marked the beginning of LayerIt development.

3.3 LayerIt!

3.3.1 Problem Definition

Since this thesis started with a focus on beat annotation, we developed the tool with a beat visualization use case in mind. However, we think that the developed program could be useful to be integrated for other MIR purposes, beat annotation and visualization were just our starting points. By itself the codebase also does not have any user interaction provided, we focused on the visualization task. Yet, the codebase was designed with future development/integration in mind. A GUI or a web-based solution being examples of possible future iterations or projects using LayerIt.

3.3.2 Proposed Solution

It was only mid April 2026 that LayerIt started being developed and is the project that is the focus of this thesis. LayerIt, is intended to be a Python OO tool for visualizing recorded performances synchronized with the corresponding music score in a time-aligned manner. This tool allows for custom made visualizations of audio recordings where visual elements are treated as independent layers that can be copied, moved and combined in order to develop custom visualizations.

3.3.3 LayerIt Overview

The project bases itself in librosa and matplotlib to generate audio-visualizations and data plots. The score used for the symbolic representation layer is rendered through ScoreWarp. The end result is an SVG file that overlays the audio-visualization, data layers and score.

The codebase integrates BeatThis the as-of-now SOTA BT algorithm. Through it's implementation we demonstrate how a given BT model can be integrated to our system in order to produce beat data visualizations.

3.3.4 Choices Made and Why

LayerIt relies on matplotlib for audio and data representations where, the SVG format is the end visual format. The reason, stems from the fact that Verovio (and subsequently ScoreWarp) render the score in SVG, and because matplotlib does provide seamless integration of SVG charts, it is more reliable to export matplotlib visuals into a SVG framework, then the other way around.

The SVG format, is a vector-based one, therefore, data that can be mathematically represented in cartesian plots (i.e., X/Y graphs) can be exported without losing resolution. This is the case for most of MIR algorithmic data visualizations i.e., continuous curves and time based annotations. However, some data types, cant be presented in this way noticeably, heatmap-based plots like spectrograms. For interaction proposes these usually are rendered in real-time, allowing for seamless navigation (i.e., zooming and panning). This is not possible since SVG is an inherently static format, therefore plots like these need to be rasterized before being integrated into the SVG. To circumvent this, we employed a way to render plots as PNG's as well. This trade-off means

that cartesian data and the score can be zoomed in infinitely while always preserving the highest level of fidelity and accuracy. On the other hand rasterized layers are bound to lose quality with zooming while, at the same time, also requiring heavier processing and memory usage.

That being said, because SVG is a text markup format, it can be easily accessed and modified, allowing it to be an intermediate format that can bridge different codebases independently of their base language (i.e., Python, JS, C++ etc...). This also serves as a crucial value argument for this choice, this being, the development and integration of LayerIt into future MIR applications.

Our main goal is that, this codebase may be easily integrated into other relevant projects to generate visualizations. Hence why we developed it as a Python Based tool that is based on commonly used MIR libraries such as librosa and matplotlib. This way, community integration and development is eased.

3.4 Chapter Summary

SHIT AI TEXT TO A REESCREVER In this chapter, we presented the different iterations that the development phase of this thesis undertook. This should contextualize the change in scope and motivation of the thesis. We then presented LayerIt, the core project of this thesis, providing an overview of its purpose, scope, architecture and design choices. One of the main arguments of this thesis is that LayerIt also provides a framework that can support modern MIR visualizations, while offering an architecture designed for future integration with other MIR applications.

Next we will explore LayerIt in more depth, providing a detailed overview of the codebase architecture and how the visualizations are generated. We will also explore how to use the codebase to generate custom visualizations, and how to integrate it with other tools and workflows.

Chapter 4

LayerIt!

In this chapter, we will explore more in depth the LayerIt, the core project of this thesis. We will explore the developed architecture of this codebase, along with presenting the required pre-requisites that are needed for generating the visualizations and providing broad clarifications on how the system works. By the end of this chapter, the reader should better understand the value of the developed approach, as well as how it can be further developed and integrated with other systems.

4.1 Layer on Layers within Layers

At its core, LayerIt provides a way to combine aligned visualizations of algorithmic data, audio and symbolic music representations (i.e., scores) that share the same timeline. The true value of our approach comes from the Python based OO layer-based methodology we employed in our approach. This allows not only the visualizations to be deeply customizable, but also, for implementing with other tools and workflows.

The visualizations generated by LayerIt follow a “layers upon layers, within layers” philosophy: visual elements are self-contained components that can be composed into upper-level elements, which in turn can be composed into yet higher levels. ?? demonstrates this methodology.

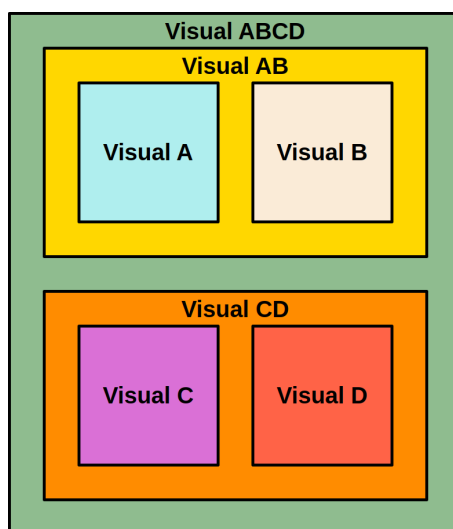


Figure 4.1: Composition of LayerIt visualization system. Each colored box represents a distinct visual element that compose higher-level visualizations.

This “layers within layers” design philosophy is applied at three different levels within the project’s codebase: the **visual level**, where visualizations are composed of aligned layered elements; the **SVG level**, in how LayerIt organizes the XML structure of the SVG (the visual output format); to the **code level**, with it’s OO architecture of the codebase. The strength of this layer-based approach lies in the flexible foundation it provides, leaving the codebase open to future development and integration with other tools, workflows, and projects.

4.2 Generating Visuals

LayerIt was developed with the goal of generating visualizations for BA and BT tasks, therefore, the current version focuses on tools for these purposes. Hence why we focus so much in providing tools for audio-to-score and data alignment tools, since we believe that these are core gaps for annotation and evaluation tasks.

File	Description
.wav	File with the audio recording of the performance
.mei	XML Score of the musical piece
.maps.json	Contains data that links onsets with score elements of the .mei
.svg	The score in visual format generated by ScoreWarp (requires .mei and .maps <i>a priori</i>)
.npz	File with algorithmic BT data

4.3 Layer Creation Logic

4.4 Conclusion

One of the main arguments of this thesis is that LayerIt also provides a framework that can support modern MIR visualizations, while offering an architecture designed for future integration with other MIR applications.

Chapter 5

Conclusions and Future Work

5.1 Achievements

5.2 Future Work